

File Handling

What is a File?

- A file is a collection of related data stored permanently on a storage device such as a Hard Disk (HDD), SSD, Pen Drive, or Memory Card.
- A file allows us to save information even after the program has finished running.

A file is a named location on secondary storage used to store data permanently.

RAM

Temporary storage

Very Fast

Data disappears when power is off

Used while program runs

File

Permanent storage

Slower than RAM

Data remains after shutdown

Used to store data permanently

Types of Files

Python mainly works with two types of files.

Text File

- A text file stores information in the form of readable characters.
- Humans can easily read it using editors like
- Notepad VS Code Sublime Text PyCharm

Characteristics

- Human readable
- Stores characters
- Uses encoding
- Easy to edit

Binary File

- Binary files store data as raw bytes.
- Humans cannot read binary files directly.

Characteristics

- Stores bytes
- Fast
- Compact
- Not human readable

Text File

Stores characters

Human readable

Uses encoding

Easy to edit

Examples: `.txt`, `.csv`, `.json`

Binary File

Stores bytes

Not human readable

No character encoding

Difficult to edit

Examples: `.jpg`, `.png`, `.pdf`, `.mp4`

What is File Handling?

File Handling is the process of performing operations on files.

These operations include

- Creating a file
- Opening a file
- Reading data
- Writing data
- Updating data
- Closing a file

Steps in File Handling

Every file operation follows the same sequence.

Open File → Read / Write → Close File

This is known as the File Handling Cycle.

`f = open("student.txt")` → `open()` returns a file object.

Perform Operations Once opened.

Close the File

Because, Python releases

- Memory
- Buffer
- Operating System resource

What Happens If You Don't Close a File?

- Memory leak
- File lock
- Unsaved data
- Buffer not flushed
- Resource wastage

The with Statement

Python provides a better way to work with files.

with open("data.txt") as f:

```
    data = f.read()
```

Python automatically closes the file.

Even if an exception occurs,

Python automatically closes the file.

Normal `open()`

`with open()`

Must call `close()` manually

Automatically closes the file

Risk of forgetting `close()`

No risk

More code

Less code

Less safe

Safer

Not recommended for beginners

Recommended and widely used

File Modes in Python

We learned how to open a file using `open()`. But opening a file is not enough.

Python also needs to know what operation you want to perform on the file.

For example:

- Read the file?
- Write to the file?
- Add new data?
- Create a new file?
- Read and Write together?
- This is done using File Modes.

What is a File Mode?

A File Mode tells Python how a file should be opened and what operations are allowed on that file.

```
f = open("student.txt", "r")
```

Types of File Modes

Python provides several file modes.

File Modes

- r, w, a, x, r+, w+, a+
- rb, wb, ab, rb+, wb+, ab+

PYTHON FILE MODES

Mode	Why use it ?	File must exist ?	Creates File ?	Read ?	Write ?	Deletes old data ?	Pointer position	Error (if any)	Example
r	Read an existing file	Yes	No	Yes	No	No	Beginning	FileNotFoundError (if file doesn't exist)	open("data.txt", "r")
w	Write new data by replacing old data	No	Yes	No	Yes	Yes	Beginning	No error	open("data.txt", "w")
a	Add new data at the end of file	No	Yes	No	Yes	No	End	No error	open("data.txt", "a")
x	Create a new file only	No	Yes	No	Yes	No	Beginning	FileExistsError (if file exists)	open("data.txt", "x")
r+	Read and update an existing file	Yes	No	Yes	Yes	No	Beginning	FileNotFoundError (if file doesn't exist)	open("data.txt", "r+")
w+	Read and write after replacing old data	No	Yes	Yes	Yes	Yes	Beginning	No error	open("data.txt", "w+")
a+	Read and append new data	No	Yes	Yes	Yes	No	End	No error	open("data.txt", "a+")
rb	Read binary files	Yes	No	Yes	No	No	Beginning	FileNotFoundError (if file doesn't exist)	open("img.jpg", "rb")
wb	Write binary files by replacing old data	No	Yes	No	Yes	Yes	Beginning	No error	open("img.jpg", "wb")
ab	Append binary data	No	Yes	No	Yes	No	End	No error	open("video.mp4", "ab")
rb+	Read and update binary files	Yes	No	Yes	Yes	No	Beginning	FileNotFoundError (if file doesn't exist)	open("img.jpg", "rb+")
wb+	Read and write binary files after replacing old data	No	Yes	Yes	Yes	Yes	Beginning	No error	open("img.jpg", "wb+")
ab+	Read and append binary data	No	Yes	Yes	Yes	No	End	No error	open("video.mp4", "ab+")

Key Points : r → read only.

- w → write (overwrite).
- a → append (add at end).
- x → exclusive creation (new file only).

+ → read and write
b → binary file

Always close the file after use or use "with open() as f".

CHAPTER 6 - READING FILES IN PYTHON

Introduction :

Reading a file means retrieving the data stored inside a file into a Python program.

Python provides several methods to read a file.

Ways to read a file in Python :

1. read()
2. read(size)
3. readline()
4. readlines()
5. for loop (Best for large files)

Sample File : student.txt

Rahul
Aman
Priya
Rohan
Niki

① read()

Definition : read() reads the entire contents of a file and returns it as a single string.

Syntax :

file.read()

Internal Working :

```
f = open("student.txt", "r")
data = f.read()
print(data)
f.close()
```

File
.....
.....

Read
Everything

One String

Python
Program

Output :

Rahul
Aman
Priya
Rohan
Niki

What is returned?

type(data)

Output : <class 'str'>

Advantages :

- Reads the entire file.
- Very easy to use.
- Good for small files.

Disadvantages :

- Loads the entire file into RAM.
- Not suitable for large files (100 MB, 1 GB, etc.)

Use Case : Use read() when file is small and you need all data at once.

② read(size)

Definition : read(size) reads only a specified number of characters from the file.

Syntax :

file.read(size)

size → number of characters to read.

Example 1 :

```
f = open("student.txt", "r")
print(f.read(6)) # reads first 6 characters
f.close()
```

Output :

Rahul

Example 2 :

```
f = open("student.txt", "r")
print(f.read(5)) # Rahul
print(f.read(5)) # Aman
f.close()
```

Internal Working :

File
Rahul
Aman
Priya
.....

Read First
size
characters

Move Pointer
Forward

Next read()
continues from
new position

③ readline()

Definition : readline() reads one line at a time from the file.

Syntax : `file.readline(size)` size → (optional) maximum characters to read.

Example 1 :

```
f = open("student.txt", "r")
print(f.readline()) # Rahul
print(f.readline()) # Aman
f.close()
```

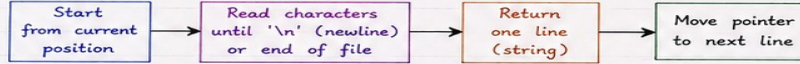
Output :

```
Rahul
Aman
```

Example 2 (with size) :

```
f = open("student.txt", "r")
print(f.readline(3)) # Rah
print(f.readline(4)) # ul\nA
f.close()
```

Internal Working :



Note : Newline character '\n' is included at the end of each line (except last line if file does not end with newline).

④ readlines()

Definition : readlines() reads all lines of the file and returns them as a list. Each element of the list is one line.

Syntax : `file.readlines()`

Example :

```
f = open("student.txt", "r")
data = f.readlines()
print(data)
f.close()
```

Output :

```
['Rahul\n',
'Aman\n',
'Priya\n',
'Rohan\n',
'Niki\n']
```

Type Returned :

<class 'list'>

Advantages

- Easy to process line by line.
- Returns list of lines.

Disadvantages

- Loads entire file into memory.

⑤ for loop (Best for large files)

Example :

```
with open("student.txt", "r") as f:
    for line in f:
        print(line.strip()) # remove \n
```

Output :

```
Rahul
Aman
Priya
Rohan
Niki
```

Why is it best ?

- Reads file line by line (one line in memory at a time).
- Very memory efficient.
- Fast even for huge files.

Comparison of Reading Methods

Method	Returns	Reads Whole File	Memory Usage	Best For
read()	String	Yes	High	Small files
read(size)	String	No (only given size)	Low	Large files (chunk by chunk)
readline()	String (one line)	No (one line at a time)	Very Low	Reading line by line
readlines()	List of Strings	Yes	High	When you need all lines as a list
for line in file	One line at a time	No	Very Low	Large files (Recommended)

Common Mistakes

- After reading entire file once, next read() returns '' (empty string).
- Forgetting to close the file (use with statement).
- Not using strip() → extra blank lines while printing.
- Using read() on very large files → high memory usage.
- Expecting readlines() to return string (It returns list).

Best Practices

- * Use with open(...) to automatically close the file.
- * Use for line in file for large files.
- * Use read(size) when processing in chunks.
- * Use strip() to remove newline characters.
- * Choose the right method based on file size and requirement.

Interview Questions

1. What is the difference between read() and readlines()?
2. Which method is best for large files?
3. What does readline() return?
4. Why does the second read() return empty string?
5. Which method returns a list?

* — CHAPTER 7 - WRITING FILES IN PYTHON — *

Introduction :

Writing to a file means sending data from a Python program and storing it permanently in a file.
Python allows us to create a new file or modify an existing file using different write operations.

File Modes used for Writing

- **w** → Write (overwrite)
- **a** → Append
- **w+** → Write and read
- **a+** → Append and read

Writing Operations in Python :

1. write()
2. Return value of write()
3. Writing multiple times
4. writelines()
5. Difference between write() and writelines()
6. Newline (\n) handling

1 write()

Definition :

write() is used to write a string to a file.

Syntax : `file.write(string)`

Internal Working :



Example :

```
f = open("demo.txt", "w")
f.write("Hello Python")
f.close()
```

After Execution :

demo.txt
Hello Python

Before Execution :

demo.txt
(File does not exist)

Important Points :

- It writes the exact string you give.
- If the file does not exist, it is created.
- If the file exists and you open in 'w' mode, old content is deleted and new content is written.

File Content

Hello Python → First write()
Good Morning → Second write()

2 Return Value of write()

write() method returns the number of characters written to the file.

Example :

```
f = open("demo.txt", "w")
count = f.write("Hello")
print(count)
# Output → 5
f.close()
```

"Hello" has 5 characters.
So write() returns 5.

Note :

It does NOT return the string written, it returns the number of characters.

3 Writing Multiple Times

Example :

```
f = open("demo.txt", "w")
f.write("Hello\n") # First write
f.write("Python\n") # Second write
f.write("Have a nice day\n") # Third write
f.close()
```

After Execution :

demo.txt
Hello
Python
Have a nice day

Explanation :

Each write() call appends the new string at the current position (end of the file pointer).

Key Point : When you open a file in 'w' mode, the file pointer is at the beginning. All write() operations start from the beginning. Old data is removed.

CHAPTER 7 - WRITING FILES IN PYTHON

Introduction :

Writing to a file means sending data from a Python program and storing it permanently in a file. Python allows us to create a new file or modify an existing file using different write operations.

File Modes used for Writing

- **w** → Write (overwrite)
- **a** → Append
- **w+** → Write and read
- **a+** → Append and read

Writing Operations in Python :

1. write()
2. Return value of write()
3. Writing multiple times
4. writelines()
5. Difference between write() and writelines()
6. Newline (\n) handling

1 write()

Definition :

write() is used to write a string to a file.

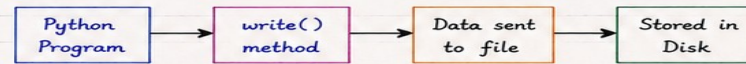
Syntax :

```
file.write(string)
```

Example :

```
f = open("demo.txt", "w")
f.write("Hello Python")
f.close()
```

How it work :



Before Execution :

demo.txt

(File does not exist)

After Execution :

demo.txt

Hello Python

Important Points :

- It writes the exact string you give.
- If the file does not exist, it is created.
- If the file exists and you open in 'w' mode, old content is deleted and new content is written.

File Content (Multiple write())

```
Hello
Python
Good Morning
```

- First write()
- Second write()
- Third write()

2 Return Value of write()

write() method returns the number of characters written to the file.

Example :

```
f = open("demo.txt", "w")
count = f.write("Hello")
print(count) # Output → 5
f.close()
```

"Hello" has
5 characters.
So write()
returns 5.

3 Writing Multiple Times

Example :

```
f = open("demo.txt", "w")
f.write("Hello\n") # First write
f.write("Python\n") # Second write
f.write("Have a nice day!\n") # Third write
f.close()
```

After Execution (demo.txt) :

```
Hello
Python
Have a nice day!
```

Explanation :

Each write() call appends the new string at the current position (end of file pointer).

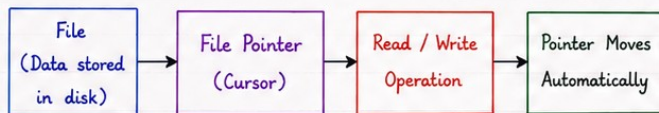
Key Point : When you open a file in 'w' mode, the file pointer is at the beginning. All write() operations start from the beginning. Old data is removed.

1. What is a File Pointer (Cursor)?

- A file pointer (cursor) is a position indicator inside a file. a file.
- It keeps track of the current position where the next read or write operation will occur.
- When we open a file, the pointer is always at the beginning (position 0).

Why it is needed?

- To know where we are in the file.
- To read or write from a specific position.
- To move to any position and process data.

Internal Working (Visual Diagram)Example File : data.txt

H	e	l	l	o	\n	W	o	r	l	d	\n
---	---	---	---	---	----	---	---	---	---	---	----

Index : 0 1 2 3 4 5 6 7 8 9 10 12



Pointer starts at index 0 when file is opened.

2. tell()

Definition : tell() returns the current position of the file pointer.

Syntax : `file.tell()`

How it works :

It returns the number of bytes (characters) from the beginning of the file to the current pointer position.

Example :

with open("data.txt", "r") as f:

print(f.tell()) # Output → 0

f.read(5)

print(f.tell()) # Output → 5

f.read(2)

print(f.tell()) # Output → 7

data.txt

Hello\nWorld

0 1 2 3 4 5 6 7 8 9 10 11

↑

Pointer after
reading 5 chars
(Hello)

Return Value :

Returns an integer representing the current position.

Use Cases :

- To check current position in file.
- Useful in debugging and file processing.

3. seek()

Definition : seek() is used to move the file pointer to a specified position.

Syntax : file.seek(offset, whence)

- offset → Position to move.
- whence → 0 → From beginning (default)
1 → From current position
2 → From end of file

How it works ?

It changes the position of the file pointer to the given offset based on whence.

Examples :

1. Move to beginning

```
f.seek(0) # Pointer at start
```

2. Move to any position

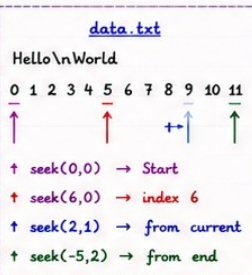
```
f.seek(10) # Pointer at index 10
```

3. Move from current position

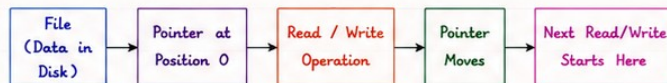
```
f.seek(5, 1) # Move 5 bytes forward
```

4. Move from end

```
f.seek(-5, 2) # Move 5 bytes backward from end
```



6. Internal Working (How it works)



- When file is opened, pointer is at position 0.
- After every read() or write(), pointer moves forward.
- Next operation starts from the new position.
- We can move pointer manually using seek().

7. Common Mistakes

1. Reading entire file once, then trying to read again.

```
f = open("data.txt", "r")
print(f.read()) # Reads everything
print(f.read()) # "" (empty string)
```

Reason : Pointer is already at end of file.

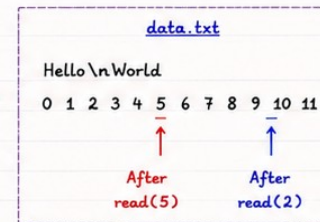
4. Relationship Between read() and File Pointer

- Whenever we read data, the pointer moves forward by the number of characters read.

Example :

with open("data.txt", "r") as f:

```
print(f.tell()) # 0
data = f.read(5) # Read 5 chars → "Hello"
print(f.tell()) # 5
data2 = f.read(2) # Read 2 chars → "\nW"
print(f.tell()) # 7
```



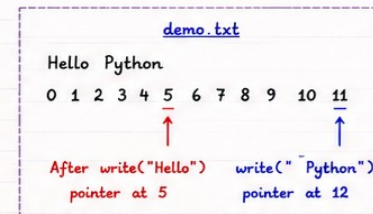
5. Relationship Between write() and File Pointer

- Whenever we write data, the pointer moves forward by the number of characters written.

Example :

with open("demo.txt", "w") as f:

```
print(f.tell()) # 0
f.write("Hello") # Write 5 chars
print(f.tell()) # 5
f.write(" Python") # Write 7 chars
print(f.tell()) # 12
```



8. Best Practices

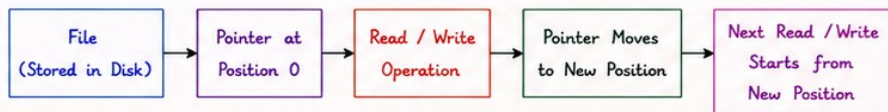
- Always use tell() to know current position.
- Use seek() to move pointer when required.
- Use seek(0) before reading the file again.
- Do not assume pointer is always at the beginning.
- Understand pointer movement for efficient file handling.
- Use pointer functions for debugging and advanced processing.

Quick Facts

- Pointer starts at position 0 when file is opened.
- read() / write() moves pointer forward.
- tell() → Gives current position.
- seek() → Moves pointer to a specific position.
- Pointer never goes backward automatically.
- Use seek() to change position manually.

6. How it works (Internal Working)

The file pointer keeps track of the current position in the file.



Key Idea :

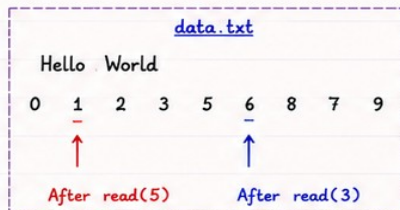
Pointer tells Python where the next read or write operation will happen.

7. Examples - See Pointer in Action

* (a) Using read()

with open("data.txt", "r") as f:

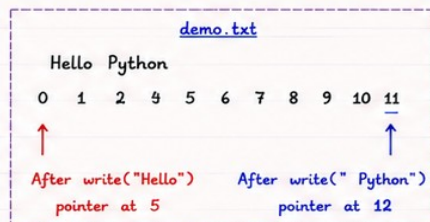
```
print(f.read(5))    # Hello
print(f.tell())     # 5
print(f.read(3))    # Wo
print(f.tell())     # 7
```



* (b) Using write()

with open("demo.txt", "w") as f:

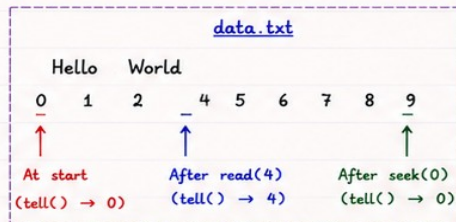
```
f.write("Hello")
print(f.tell())    # 5
f.write(" Python")
print(f.tell())    # 12
```



* (c) Using seek() and tell()

with open("data.txt", "r") as f:

```
print(f.tell())    # 0
f.read(4)          # Hell
print(f.tell())    # 4
f.seek(0)
print(f.tell())    # 0
print(f.read())    # Hello\nWorld
```



8. Common Mistakes

- 1. Reading entire file once, then trying to read again.

```
f = open("data.txt", "r")
print(f.read())    # Reads everything
print(f.read())    # "" (empty string)
```

Reason : Pointer is already at end of file.
- 2. Expecting pointer to be at beginning again.
Pointer does not go back automatically.
Use seek(0) to move it to start.
- 3. Not using seek() before reading again.
Without seek(0), next read will start from current position.
- 4. Incorrect use of seek() positions.
Passing negative or large invalid positions may cause errors.

9. Best Practices

- ✓ Always use tell() to know current position.

```
pos = f.tell()
```
- ✓ Use seek() to move pointer when needed.

```
f.seek(0)    # Move to beginning
```
- ✓ Use seek(0) before reading the file again.

```
f.seek(0)    f.read()
```
- ✓ Do not assume pointer is always at the beginning. It changes after every read() or write().
- ✓ Understand pointer movement for efficient file handling and debugging.
- ✓ Use pointer functions (tell(), seek()) in advanced file processing.